



“Automated Refactoring Demonstration”

Submitted by:

Mahir Faisal | BSSE 1316

Submitted to:

Toukir Ahammed
Lecturer

**Institute of Information Technology,
University of Dhaka**

Submitted on 26th November, 2025

Automated Refactoring Demonstration.....	3
Github Copilot.....	3
1. Cleaning Up Repeated Code.....	3
2. Making Code More Concise.....	6
Eclipse.....	7
Extract Method.....	7

Automated Refactoring Demonstration

This document demonstrates the use of automated refactoring tools provided by IDEs.

Github Copilot

Two sample cases were explored: 1) Cleaning up repeated code 2) Making code more concise.

1. Cleaning Up Repeated Code

Before Refactoring:

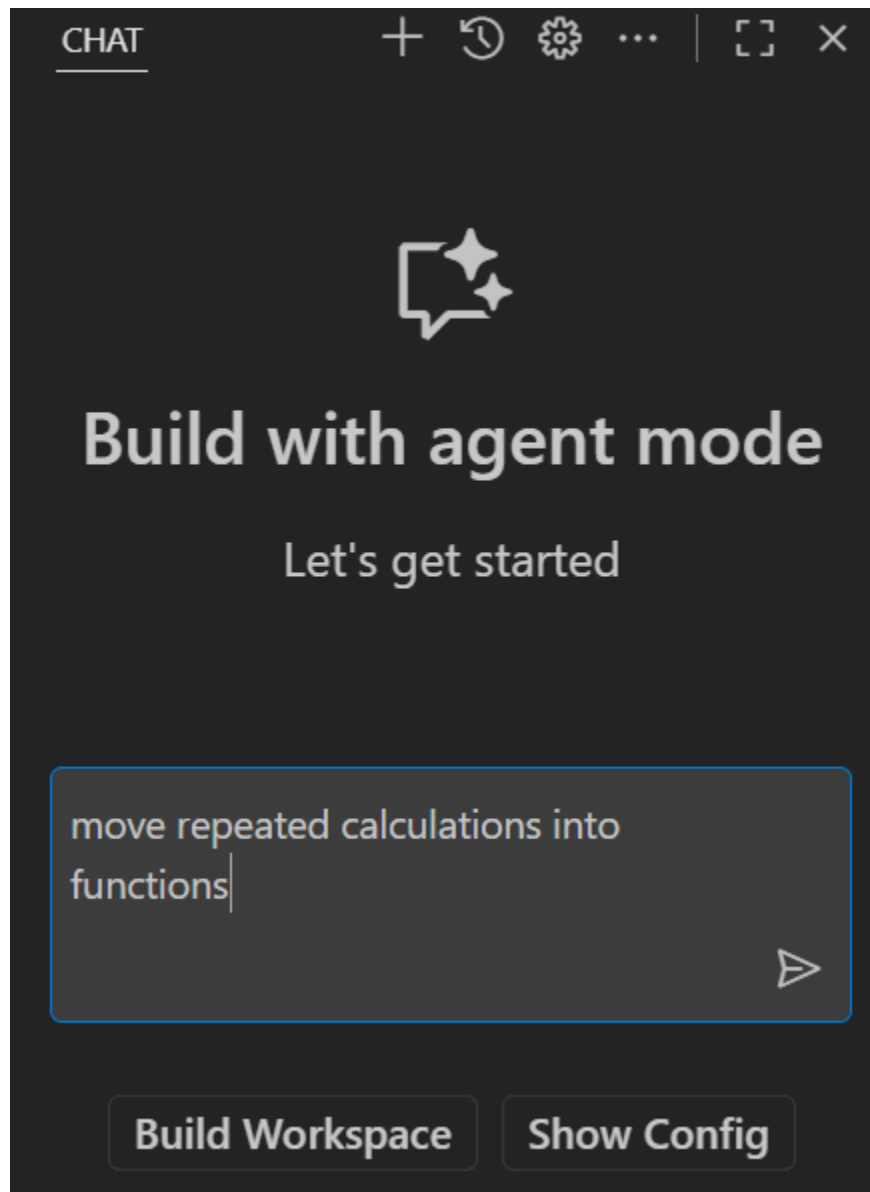
```
smell_code.py > ...
1  total_sales = 0
2
3  apple_price = 3
4  apples_sold = 100
5  total_sales += apple_price * apples_sold
6
7  orange_price = 5
8  oranges_sold = 50
9  total_sales += orange_price * oranges_sold
10
11 print(f"Total: {total_sales}")
12
```

PROBLEMS OUTPUT TERMINAL ...

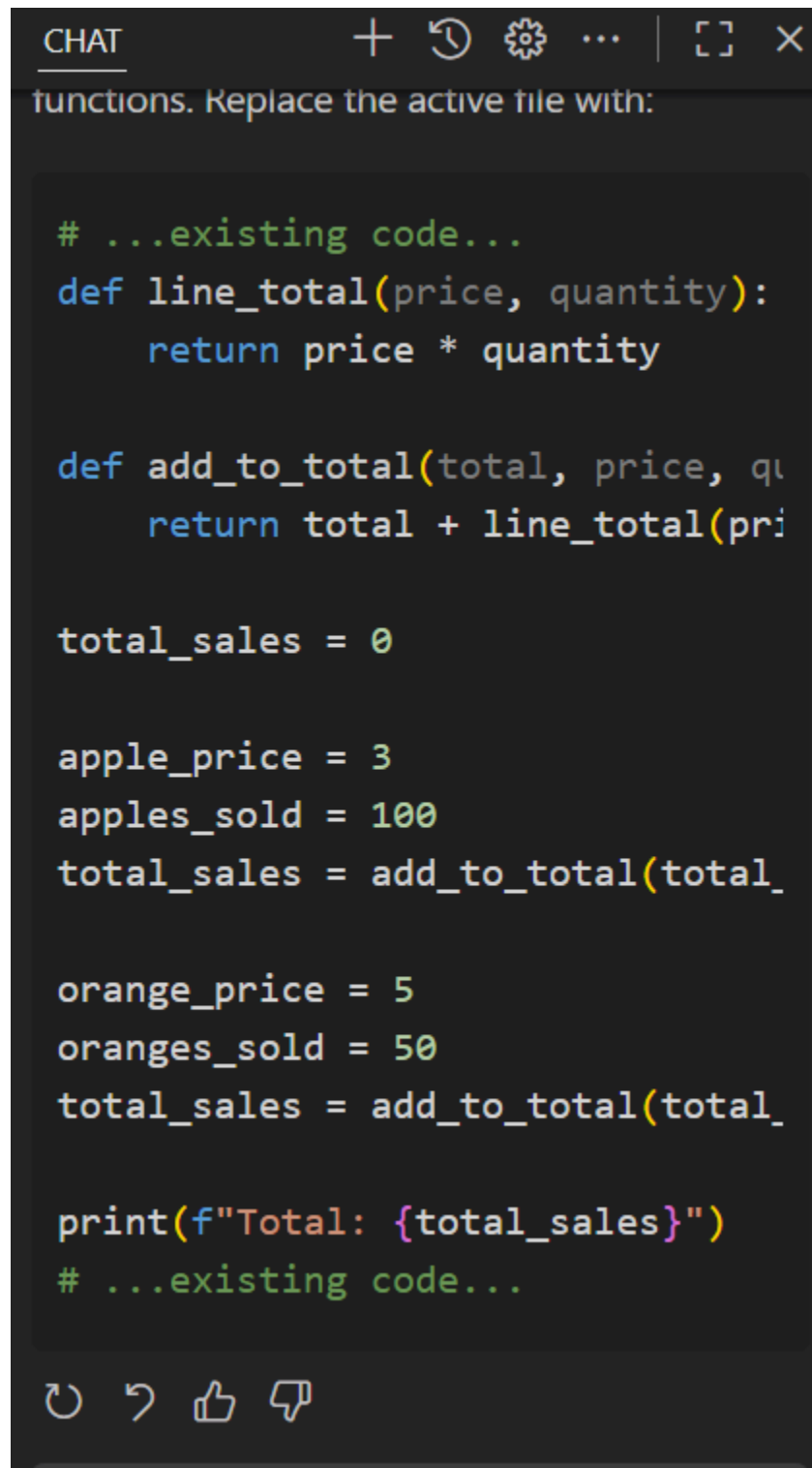
PS D:\8th\Software Maintenance\Refactoring_provided_by_IDE> python .\smell_code.py
Total: 550

PS D:\8th\Software Maintenance\Refactoring_provided_by_IDE>

Refactoring Prompt:



After Refactoring:



```
functions. Replace the active file with:

# ...existing code...
def line_total(price, quantity):
    return price * quantity

def add_to_total(total, price, quantity):
    return total + line_total(price, quantity)

total_sales = 0

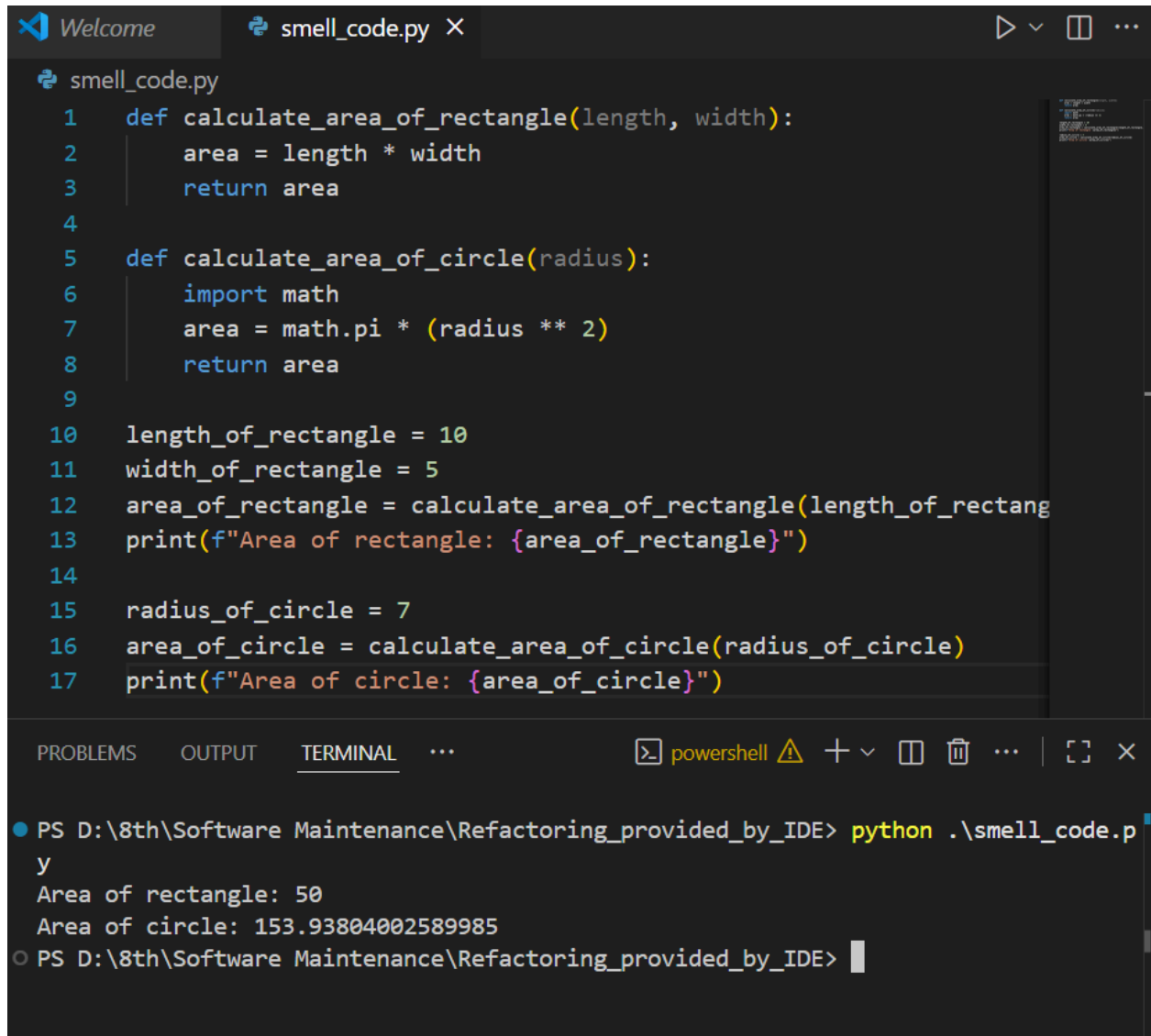
apple_price = 3
apples_sold = 100
total_sales = add_to_total(total_sales, apple_price, apples_sold)

orange_price = 5
oranges_sold = 50
total_sales = add_to_total(total_sales, orange_price, oranges_sold)

print(f"Total: {total_sales}")
# ...existing code...
```

2. Making Code More Concise

Before Refactoring:

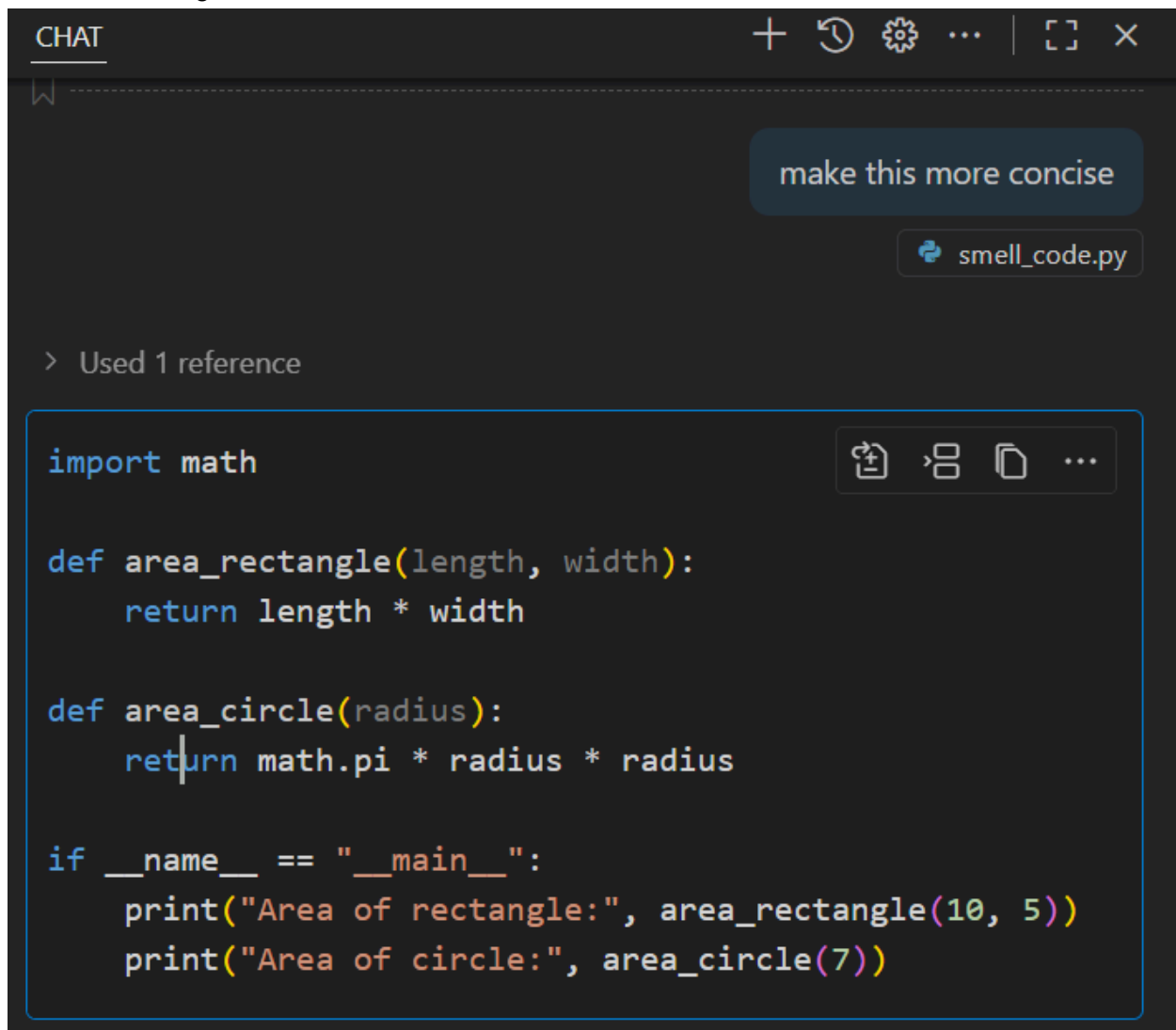


```
smell_code.py
1 def calculate_area_of_rectangle(length, width):
2     area = length * width
3     return area
4
5 def calculate_area_of_circle(radius):
6     import math
7     area = math.pi * (radius ** 2)
8     return area
9
10 length_of_rectangle = 10
11 width_of_rectangle = 5
12 area_of_rectangle = calculate_area_of_rectangle(length_of_rectangle, width_of_rectangle)
13 print(f"Area of rectangle: {area_of_rectangle}")
14
15 radius_of_circle = 7
16 area_of_circle = calculate_area_of_circle(radius_of_circle)
17 print(f"Area of circle: {area_of_circle}")
```

PROBLEMS OUTPUT TERMINAL ... powershell ⚠ + ▾ 🗑 ... | 🔍 ✕

```
● PS D:\8th\Software Maintenance\Refactoring_provided_by_IDE> python .\smell_code.py
Area of rectangle: 50
Area of circle: 153.93804002589985
○ PS D:\8th\Software Maintenance\Refactoring_provided_by_IDE>
```

After Refactoring:



The screenshot shows a chat window in an IDE. The title bar says 'CHAT' and has standard window controls. A bookmark icon is on the left. A button 'make this more concise' is in the top right. Below it is a file icon and the name 'smell_code.py'. A section header '> Used 1 reference' is visible. The main area contains a code block with Python code for calculating the area of a rectangle and a circle. The code is syntax-highlighted and has a light blue border. To the right of the code is a toolbar with icons for copy, paste, and other actions.

```
import math

def area_rectangle(length, width):
    return length * width

def area_circle(radius):
    return math.pi * radius * radius

if __name__ == "__main__":
    print("Area of rectangle:", area_rectangle(10, 5))
    print("Area of circle:", area_circle(7))
```

[Eclipse](#)

Extract Method

Before Refactoring:

```

1 package Smell;
2
3 public class OrderProcessor {
4     public void process() {
5         double total = 0;
6         total += priceOfItemA * countA;
7         total += priceOfItemB * countB;
8         System.out.println("Total: " + total);
9     }
10 }
11

```

After Refactoring:

```

package Smell;

public class OrderProcessor {
    public void process() {
        double total = extracted(priceOfItemA, countA, priceOfItemB, countB);
        System.out.println("Total: " + total);
    }

    private double extracted(double priceOfItemA, int countA, double priceOfItemB, int countB) {
        double total = 0;
        total += priceOfItemA * countA;
        total += priceOfItemB * countB;
        return total;
    }
}

```

These refactoring examples reduce repetition and improve clarity. Automated IDE refactoring tools can perform these optimizations quickly, ensuring cleaner, more maintainable code.